INSTITUTO TÉCNICO RICALDONE

Fundamentos de Diseño de Sistemas y Modelos SDLC

Msc. Josue A. Guinea Henríquez

1 Construyendo el Futuro: Fundamentos de Diseño de Sistemas y Modelos SDLC

Cuando una persona decide construir una vivienda, rara vez comienza apilando ladrillos sin un propósito claro. El paso lógico inicial consiste en buscar a un arquitecto, trazar planos, calcular materiales y definir el orden de construcción de cada espacio. En el mundo del software, ocurre exactamente lo mismo. Escribir código sin una planificación previa es, con frecuencia, una receta para el fracaso de un proyecto. Aquí es donde entran en juego disciplinas fundamentales como el Diseño de Sistemas y el Ciclo de Vida del Desarrollo de Software (SDLC).

A lo largo de este texto, se explorará cómo los ingenieros estructuran soluciones tecnológicas complejas y cuáles son las metodologías que permiten llevar una idea desde su concepción inicial hasta las manos del usuario final.

2 Fundamentos de Diseño de Sistemas

El Diseño de Sistemas es el proceso de definir la arquitectura, los componentes, los módulos, las interfaces y los datos de un sistema para satisfacer requisitos específicos. No se trata de qué lenguaje de programación usar, sino de cómo las piezas del rompecabezas encajan para formar una imagen coherente, robusta y eficiente.

2.1 Escalabilidad

Un sistema debe ser capaz de manejar una carga de trabajo creciente sin colapsar. Existen dos formas principales de escalar:

- **Escalabilidad Vertical (Scale-up):** Consiste en añadir más recursos (CPU, RAM) a un único servidor existente. Es como ponerle un motor más grande a tu coche; llega un punto en el que el coche no soporta un motor más pesado.
- **Escalabilidad Horizontal (Scale-out):** Implica añadir más servidores al sistema para distribuir la carga. En lugar de un coche gigante, tienes una flota de coches trabajando juntos. Es el estándar moderno para grandes aplicaciones.

2.2 Disponibilidad y Confiabilidad

Los usuarios de hoy esperan que las aplicaciones funcionen las 24 horas del día, los 7 días de la semana.

- **Confiabilidad:** Es la probabilidad de que un sistema no falle en un periodo de tiempo determinado. (¿Hace lo que debe hacer correctamente?).
- **Disponibilidad:** Es el porcentaje de tiempo que un sistema está operativo y accesible. (¿Puedo acceder a él cuando lo necesito?). Para lograr alta disponibilidad, los ingenieros utilizan la **redundancia** (tener componentes de respaldo por si la principal falla).

2.3 Acoplamiento y Cohesión

Estos son los dos pilares de un código mantenible y modular:

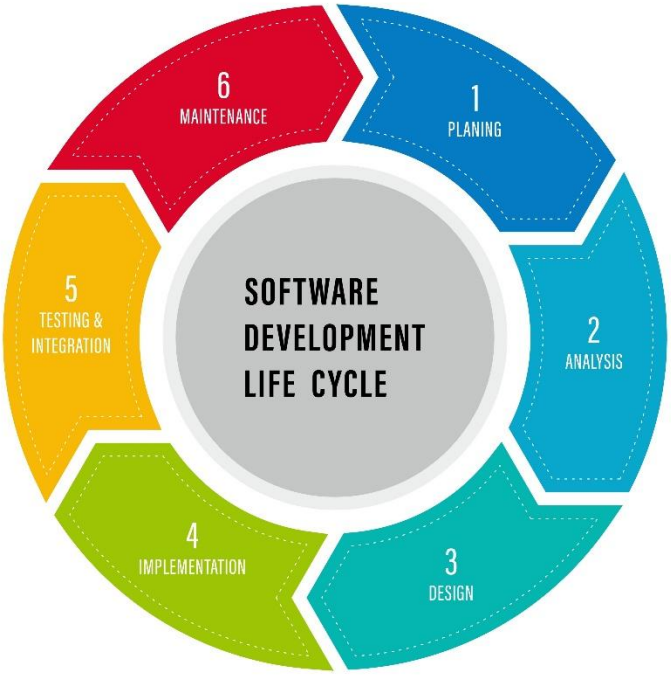
- **Cohesión (Alta es mejor):** Mide cuán relacionadas están las responsabilidades dentro de un mismo módulo. Un módulo altamente cohesivo hace una sola cosa, y la hace muy bien.
- **Acoplamiento (Bajo es mejor):** Mide el grado de dependencia entre diferentes módulos. Si cambias un componente y el sistema entero se rompe, tienes un alto acoplamiento. Buscamos módulos independientes para que actualizar el sistema sea indoloro.

3 El Ciclo de Vida del Desarrollo de Software (SDLC)

Si el Diseño de Sistemas es el plano de nuestra casa digital, el SDLC es el cronograma y el método de construcción. El SDLC es un marco estructurado que detalla los pasos necesarios para crear un software de alta calidad, a tiempo y dentro del presupuesto.

Figura 1

Fases del ciclo de vida del sistema



Fuente: [Gemini, abril 2026] Elaborado con Inteligencia Artificial

Casi todos los modelos de desarrollo comparten, en esencia, las siguientes fases universales:

1. **Análisis de Requisitos:** Consiste en descubrir qué necesita exactamente el cliente. Requiere un alto grado de empatía para entender el problema o la necesidad real del usuario.
2. **Diseño:** Define cómo se va a construir la solución. Aquí se traza la arquitectura, las bases de datos y el diseño de interfaces.
3. **Implementación (Desarrollo):** Es la fase de escritura de código, donde las ideas abstractas se traducen en un producto tangible.
4. **Pruebas (Testing):** Se verifica rigurosamente si el software funciona como debería y si cumple con los estándares de seguridad.
5. **Despliegue:** Consiste en la entrega del producto al entorno real para que los usuarios comiencen a interactuar con él.
6. **Mantenimiento:** Involucra la solución de errores imprevistos, actualizaciones y mejoras continuas. Se entiende que el software no es un producto estático, sino un ente en constante evolución.

4 Modelos SDLC Clásicos y Modernos

La forma en que los equipos de tecnología transitan por las fases mencionadas ha evolucionado drásticamente a lo largo de las décadas. A continuación, se describen los modelos más influyentes:

4.1 El Modelo en Cascada (Waterfall)

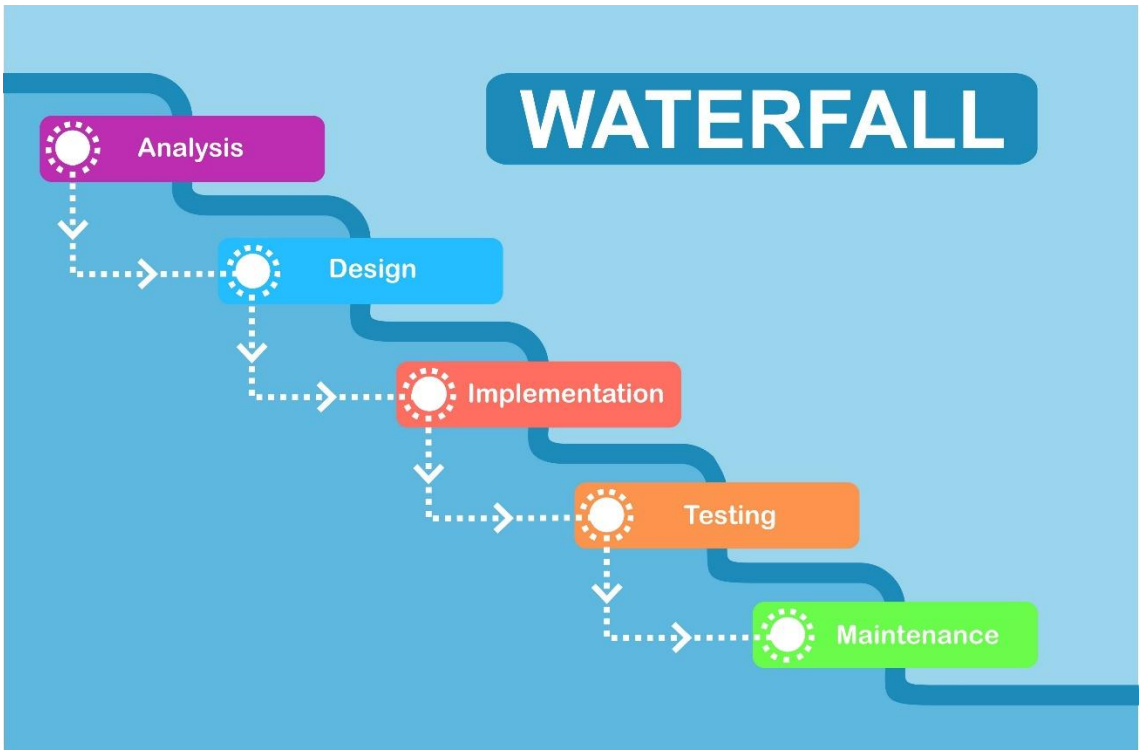
Considerado el modelo pionero, se caracteriza por ser un enfoque estrictamente lineal y secuencial. En esta metodología, no es posible avanzar a la siguiente fase sin haber completado y documentado exhaustivamente la anterior.

La perspectiva humana: Resulta reconfortante para la gestión tradicional porque ofrece una hoja de ruta sumamente clara, presupuestos exactos y un calendario predecible. Es ideal para proyectos donde los requisitos (como en el software médico o aeroespacial) son inamovibles.

La limitación: Carece de flexibilidad. Si durante la fase de pruebas se descubre un error de diseño fundamental, retroceder resulta extremadamente costoso y suele generar frustración en los equipos de trabajo.

Figura 2

Modelo en cascada Waterfall



Fuente: [Gemini, abril 2026] Elaborado con Inteligencia Artificial

El modelo en Cascada es la herramienta adecuada cuando la prioridad absoluta del proyecto es el control riguroso, la documentación exhaustiva y el cumplimiento estricto de normativas.

Circunstancias ideales para su uso:

- **Requisitos inamovibles:** El cliente sabe exactamente qué quiere desde el primer día y existe la certeza de que las necesidades no cambiarán durante el desarrollo.

- **Entornos altamente regulados:** Proyectos que requieren auditorías legales o certificaciones estrictas, donde la falta de documentación puede resultar en demandas o pérdida de licencias.
- **Baja tolerancia al fallo:** Sistemas donde un error en producción puede costar vidas humanas o pérdidas financieras catastróficas.
- **Presupuesto y tiempo fijos:** Cuando se requiere un contrato cerrado donde el costo total y la fecha de entrega exacta deben establecerse antes de escribir la primera línea de código.

Ejemplos de aplicación: Software para equipos médicos, sistemas de control de tráfico aéreo, plataformas de nómina gubernamental o sistemas bancarios centrales (Core Banking).

4.2 El Modelo Ágil (Agile)

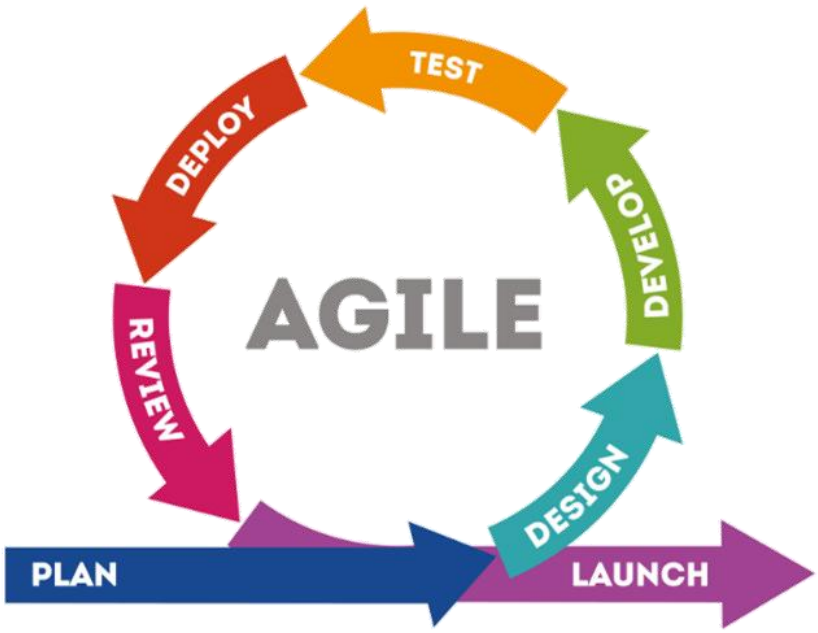
Surgió como una respuesta directa a la rigidez del modelo en Cascada. *Agile* no es una metodología única, sino una filosofía de trabajo (que engloba marcos como Scrum o Kanban) basada en la iteración constante y la adaptabilidad.

La perspectiva humana: Prioriza a las personas y sus interacciones por encima de las herramientas y los procesos rígidos. El desarrollo se divide en ciclos cortos (Sprints) que duran entre dos y cuatro semanas. Al final de cada ciclo, se entrega un fragmento de software funcional, lo que permite abrazar el cambio y mantener al cliente involucrado.

El desafío: Requiere habilidades de comunicación excepcionales, una alta disciplina de equipo y puede generar incertidumbre en los clientes respecto al costo y tiempo total a largo plazo.

Figura 3

Esquema del modelo Agile



Fuente: The University of Science & Technology MSMK (2025).

El enfoque Ágil brilla en entornos donde la incertidumbre es alta y la capacidad de reaccionar rápidamente a los cambios del mercado define el éxito o el fracaso del producto.

Circunstancias ideales para su uso:

- **Requisitos volátiles:** El cliente tiene una visión general, pero los detalles exactos del producto se irán descubriendo a medida que los usuarios reales interactúen con las primeras versiones.
- **Prioridad en el "Time-to-Market":** Existe una urgencia comercial por lanzar una versión funcional (Producto Mínimo Viable) lo antes posible para adelantarse a la competencia.
- **Innovación y experimentación:** Proyectos creativos donde el equipo necesita probar diferentes soluciones técnicas o de diseño para ver cuál genera mayor impacto.
- **Alta colaboración con el cliente:** El cliente está dispuesto y disponible para involucrarse semanalmente, revisando avances y redefiniendo prioridades.

Ejemplos de aplicación: Aplicaciones móviles de consumo, plataformas de comercio electrónico, redes sociales o software para *startups* tecnológicas.

4.3 El Modelo en Espiral (Spiral)

Este enfoque combina la naturaleza iterativa de las metodologías ágiles con los aspectos sistemáticos del modelo en Cascada, pero introduce un elemento diferenciador y crítico: el análisis de riesgos.

Funcionamiento: El proyecto avanza dibujando una forma de espiral. Cada bucle representa una fase completa de planificación, análisis de riesgos, ingeniería y evaluación. Antes de proceder a escribir el código, el equipo hace una pausa para preguntarse: *"¿Qué podría salir mal y cómo se mitiga este riesgo?"*.

La perspectiva humana: Es el modelo por excelencia para proyectos experimentales, de escala masiva o de alto riesgo, donde el impacto de un posible fracaso es real y necesita ser gestionado desde una perspectiva analítica y preventiva.

El modelo en Espiral se reserva para proyectos de una magnitud tan grande y compleja que lanzarse a programar (como en Agile) o intentar planificar todo desde el inicio (como en Cascada) resultaría irresponsable. Su núcleo es la identificación y mitigación de riesgos antes de realizar grandes inversiones.

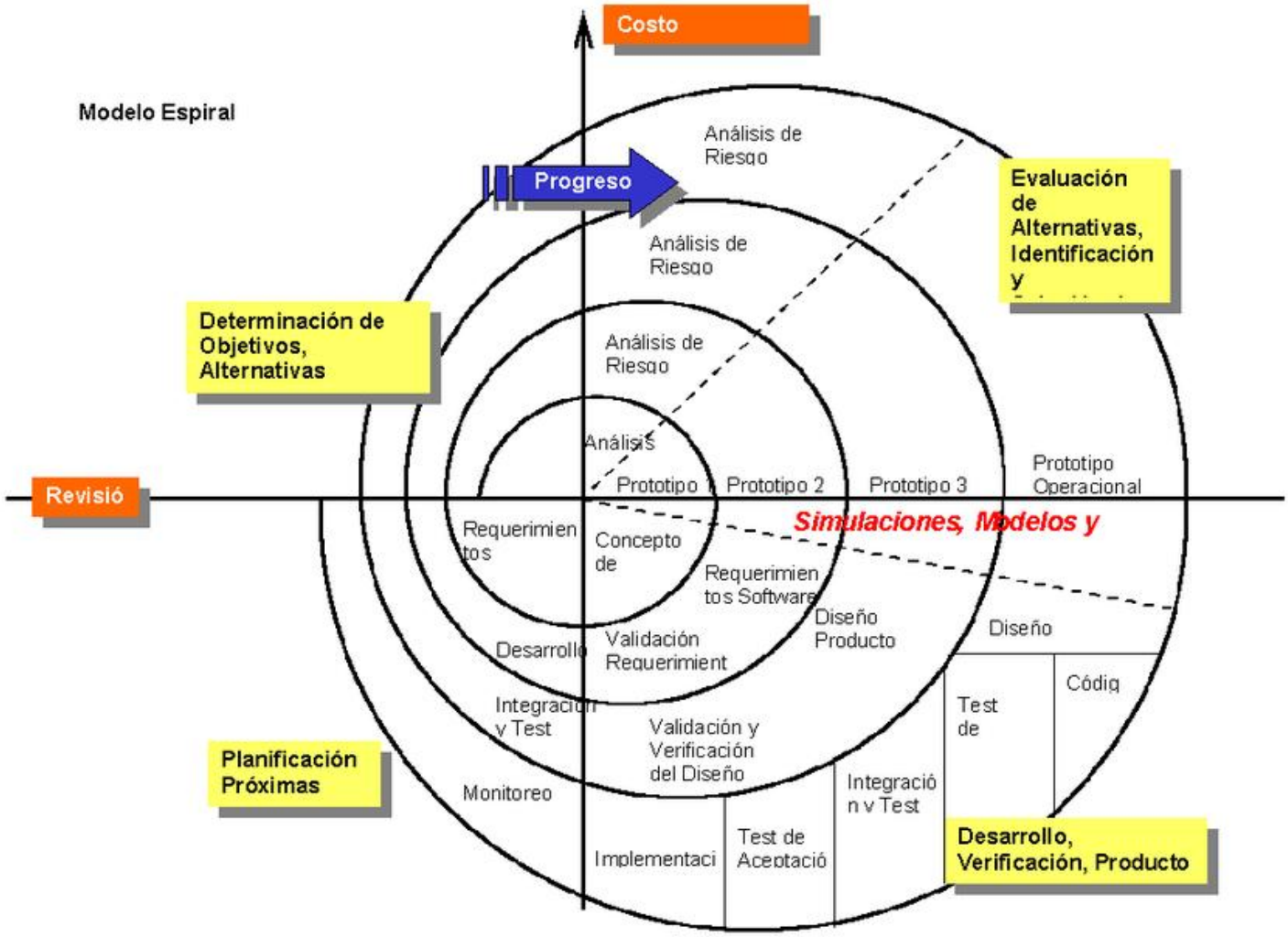
Circunstancias ideales para su uso:

- Alto riesgo técnico o financiero: Proyectos masivos donde un fracaso comprometería la viabilidad de toda la organización.
- Investigación y Desarrollo (I+D): La tecnología necesaria para construir el sistema es nueva, experimental o aún no ha sido probada a gran escala.
- Proyectos a muy largo plazo: Desarrollos que tomarán varios años y requieren validaciones profundas en cada etapa para justificar la continuidad de la inversión presupuestaria.
- Integración de sistemas complejos: Cuando el nuevo software debe conectarse de manera impecable con múltiples sistemas heredados y arquitecturas dispares.

Ejemplos de aplicación: Creación de nuevos sistemas operativos, software de exploración espacial, desarrollo de motores de videojuegos desde cero o la digitalización completa de la infraestructura a nivel nacional.

Figura 4

Esquema de modelo espiral para el desarrollo de software



Fuente: researchgate.net

Tabla 1

Resumen sobre los modelos

Factor de Decisión	Cascada (Waterfall)	Ágil (Agile)	Espiral (Spiral)
Claridad de Requisitos	Muy alta desde el inicio.	Baja; evolucionan con el tiempo.	Media; se descubren por fases.
Gestión de Riesgos	Baja (se asume que el plan es perfecto).	Media (se corrigen sobre la marcha).	Muy Alta (es el foco principal).
Flexibilidad al Cambio	Prácticamente nula.	Muy Alta.	Moderada.

Frecuencia de Entregas	Una sola entrega final.	Entregas continuas y pequeñas.	Entregas por cada ciclo importante.
Tamaño del Proyecto	Pequeño a Grande.	Pequeño a Mediano.	Masivo / Complejo.

ACTIVIDAD PRÁCTICA: ARQUITECTOS AL RESCATE DEL PROYECTO "MEDISYS"

Objetivo de Aprendizaje: Analizar un caso de estudio con requisitos contradictorios, seleccionar el modelo de Ciclo de Vida de Desarrollo de Software (SDLC) más adecuado y planificar las fases iniciales del proyecto justificando las decisiones técnicas.

El Escenario

El Ministerio de Salud ha contratado a la clase (que actuará como una consultora de software) para desarrollar un gran proyecto tecnológico llamado "MediSys". Sin embargo, el proyecto está dividido en dos módulos completamente diferentes:

- **Módulo A (Historial Clínico Central):** Un sistema centralizado que almacenará el historial médico de todos los pacientes del país. Debe cumplir con estrictas leyes de privacidad de datos, no puede tener fallos (ya que la vida de los pacientes depende de la precisión de los datos) y las reglas de negocio están dictadas por normativas gubernamentales que no cambiarán.
- **Módulo B (App Móvil de Bienestar "MediVida"):** Una aplicación móvil para que los ciudadanos registren sus hábitos de sueño, dieta y ejercicio. El Ministerio no tiene claro qué funciones serán populares, por lo que quieren lanzar una versión básica rápido, probarla con jóvenes y agregar nuevas funciones cada mes según los comentarios de los usuarios en redes sociales.

Instrucciones

La clase debe dividirse en equipos de 4 o 5 personas. Cada equipo asume el rol de "Comité de Arquitectura de Software" y debe completar los siguientes pasos:

Paso 1: Elección y Justificación del Modelo

- Analizar el Módulo A. Seleccionar un modelo SDLC (Cascada, Ágil, Espiral, etc.) para su desarrollo. Escribir un párrafo argumentando por qué es la única opción viable, mencionando los riesgos de usar un modelo incorrecto.
- Analizar el Módulo B. Seleccionar un modelo SDLC diferente al anterior. Justificar la respuesta basándose en la naturaleza cambiante del mercado de aplicaciones móviles.

Paso 2: Mapeo de Fases (Simulación)

Para el modelo elegido en el Módulo B (App Móvil), el equipo debe simular cómo se vería el primer ciclo de trabajo. Deben redactar:

- Análisis: Tres requisitos o funciones básicas que incluirían en la primera versión (MVP - Producto Mínimo Viable).
- Diseño: ¿Qué plataforma priorizarán (iOS o Android) y por qué?
- Pruebas: ¿Cómo probarán que a los usuarios les gusta la aplicación antes del siguiente ciclo?

Paso 3: Presentación de la "Autopsia de un Desastre"

Cada equipo debe preparar una breve presentación de 5 minutos. En el último minuto de la presentación, deben responder a la siguiente pregunta trampa: *"¿Qué pasaría si el Ministerio nos obliga a usar el Modelo en Cascada para la App Móvil (Módulo B)? ¿Cuáles serían las consecuencias financieras y humanas en seis meses?"*.

Entregables Requeridos

- Un documento (máximo 2 páginas) con las justificaciones del Paso 1 y el mapeo del Paso 2.
- La presentación oral (Paso 3) defendiendo sus decisiones frente a la clase (que actuará como la junta directiva del Ministerio).

Criterio	Excelente (100%)	Aceptable (75%)	Necesita Mejora (50%)
Justificación del Modelo	Argumentos sólidos basados en la teoría del SDLC, riesgos y necesidades del cliente.	Selección correcta del modelo, pero con argumentos superficiales.	Selección incorrecta del modelo o falta de justificación técnica.
Aplicación Práctica	El mapeo del Módulo B es realista y respeta las reglas del modelo seleccionado.	El mapeo es correcto, pero ignora algunas fases o principios del modelo.	Las fases descritas no corresponden al modelo de software elegido.
Pensamiento Crítico	Identifica con precisión las consecuencias catastróficas de usar el modelo incorrecto.	Menciona consecuencias generales sin profundizar en el impacto humano/financiero.	No logra explicar por qué forzar un modelo inadecuado es perjudicial.